PROJECT REPORT

Jain College of Engineering and Technology, Hubballi

Automated Sliding Window

for Ambient Air Quality

Design, Development and Validation of an ESP32-Based Smart Ventilation System

Academic Year 2025–26

Department of Electronics and Communication Engineering

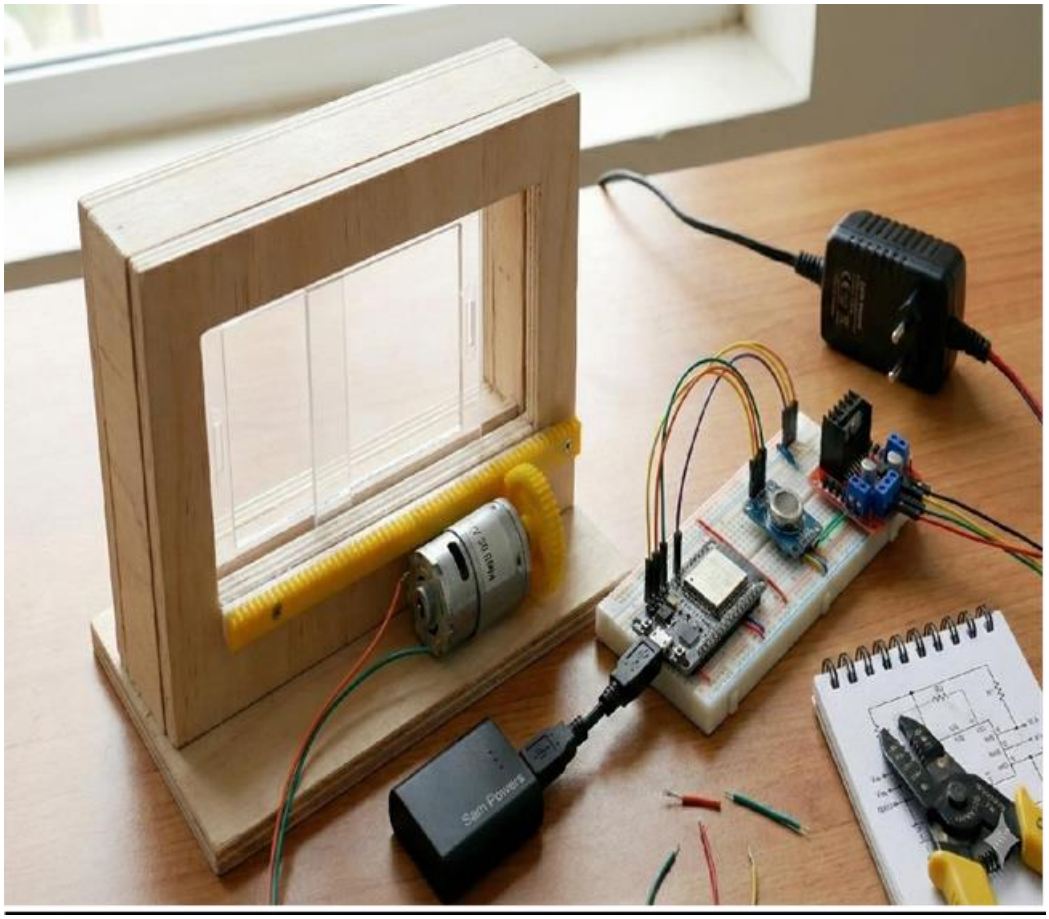
ENGINEERING TEAM

Samarth G. Ichageri — Electronics & Systems Lead

Satvik Pandurangi — Hardware & Software Engineer

Guide: **Dr**. Mahendra M. K.

PROJECT AT A GLANCE



Complete assembled prototype — ESP32, MQ sensor, L298N driver, 12V DC motor with rack-and-pinion on wooden window frame

Controller	Sensor	Motor Driver	Motor	Mechanism
ESP32	**MQ Gas Sensor**	**L298N**	**12V DC Geared**	**Rack & Pinion**
Rack Length	Pinion Diameter	Threshold	Travel Time	Stability Delay
11 inches	**3 cm**	**1900 ADC**	**6500 ms**	**10 seconds**

System Summary

An MQ-series gas sensor continuously monitors ambient air quality and feeds analog readings to an ESP32 microcontroller. When the ADC reading crosses the configured threshold of 1900 and remains above it for 10 seconds, the ESP32 drives a 12V DC geared motor through an L298N H-bridge driver to close the window via a rack-and-pinion mechanism. When clean-air conditions return and persist for 10 seconds, the motor opens the window. The complete system operates on a split 5V/12V supply and provides visual status via GPIO-driven LEDs.

01 Executive Summary

This report documents the design, development, and testing of an automated sliding window system that responds to real-time ambient air quality measurements. The system monitors pollutant concentration using an MQ-series gas sensor connected to an ESP32 microcontroller. Based on a configurable ADC threshold, the ESP32 drives a 12V DC geared motor through an L298N motor driver to open or close a rack-and-pinion actuated window panel.

The project integrates embedded systems, sensor instrumentation, and mechanical actuation into a single functional prototype. A 10-second stability delay prevents repeated actuation from short-duration sensor fluctuations, and a 6500 ms timed motor drive completes the full 11-inch rack stroke.

The prototype was assembled on a breadboard and tested under controlled laboratory conditions using incense smoke as a simulated pollutant source. Testing demonstrated reliable window actuation under both clean-air and polluted-air conditions, with the stability delay preventing false actuations during brief sensor fluctuations.

Total component cost remains under ₹1,500, making the system suitable for residential, educational, and institutional settings. The design is modular and supports future expansion including IoT monitoring, multi-gas sensing, and remote control.

02 Problem Statement

Background

Air quality in urban environments varies significantly throughout the day due to vehicular emissions, industrial activity, construction dust, and indoor sources such as cooking. Prolonged exposure to polluted air in indoor spaces contributes to respiratory irritation and long-term health effects. The World Health Organization (WHO) identifies household air pollution as a major global health risk.

Conventional window systems are manually operated and do not respond to changes in ambient air quality. A window left open during an outdoor pollution event, or kept closed during clean outdoor conditions, represents a ventilation failure with direct health consequences.

Identified Gap

Scenario	Problem	Consequence
Window open during pollution event	Polluted air enters the space	Indoor AQI degrades within minutes
Window closed during clean conditions	CO_2 accumulates indoors	Reduced air quality, occupant discomfort

Scenario	Problem	Consequence
Manual operation during sleep hours	No human response possible	Prolonged unmanaged exposure
No feedback mechanism present	Pollution events go undetected	Occupant unaware of air quality change

Problem Definition

The problem is to design an automated ventilation control system that continuously monitors ambient air quality and actuates a physical window mechanism to maintain acceptable indoor air quality, without requiring human intervention.

03 Review of Existing Solutions

Comparison of Approaches

Solution Type	Key Limitation	Typical Cost (INR)
Manual windows	Requires occupant presence and awareness	—
Scheduled HVAC systems	Operates on time schedules, not AQI data	50,000 – 5,00,000
Smart HVAC systems	High cost, complex installation, commercial-scale	1,00,000+
IoT AQI monitors	Provides readings and alerts; no physical actuation	2,000 – 20,000
Commercial auto-windows	Triggered by rain or temperature, not air quality	30,000+
This project	Full sensing-to-actuation loop at low cost	~1,200 – 1,500

Key Observation

Existing low-cost IoT solutions provide air quality monitoring but do not close the loop with physical actuation. High-cost automated systems exist in commercial HVAC, but are not suitable for individual windows at residential or classroom scale. This project addresses that gap by integrating sensing and actuation in a single, low-cost embedded system.

> **Design Decision: Local Processing Only**
> The prototype runs all decision logic on the ESP32 without cloud connectivity. This eliminates network latency from the actuation loop and ensures the system operates even without internet access. Cloud features are planned for a future revision.

04 Solution Overview

System Description

The proposed system uses an MQ-series gas sensor to measure ambient pollutant concentration. The sensor output is read as an analog voltage by the ESP32 microcontroller via its built-in 12-bit ADC. The microcontroller compares this reading against a preconfigured threshold value of 1900 ADC units. If the reading exceeds the threshold and remains above it for 10 seconds, the system closes the window. If the reading falls below the threshold for 10 seconds, the window opens.

Operating States

State	Trigger Condition
Window opens	ADC reading < 1900 — condition persists for 10 seconds
Window closes	ADC reading ≥ 1900 — condition persists for 10 seconds
State held	Condition does not persist beyond the stability gate
Motor stops	6500 ms of active drive elapsed; IN1 and IN2 set LOW

Signal Flow

```
MQ Gas Sensor → ESP32 ADC (GPIO 34) → Threshold Comparison → 10 s
Stability Timer → L298N Motor Driver → 12V DC Motor → Rack-and-Pinion
→ Sliding Window Panel
```

Note:
The 10-second stability delay is a software-controlled gate that prevents actuation from brief sensor spikes. The window moves only when a threshold crossing is sustained, which significantly reduces mechanical wear over the system lifetime.

05 System Architecture

Layered Component Overview

Layer	Component	Function
Sensing	MQ Gas Sensor	Converts pollutant concentration to proportional voltage (0–5 V)
Signal Conditioning	ESP32 ADC — GPIO 34	Converts 0–3.3 V input to 12-bit digital value (0–4095)
Processing	ESP32 Firmware (FSM)	Threshold comparison, stability timing, state management
Control	L298N H-Bridge Module	Drives 12V motor forward/reverse based on direction signals
Actuation	12V DC Geared Motor (30 RPM)	Produces rotational torque coupled to the pinion gear
Mechanical	Rack and Pinion Assembly	Converts motor rotation to 11-inch linear window displacement
Indication	Blue LED (GPIO 18), Red LED (GPIO 19)	Visual status feedback for open/closed window state
Power — Logic	5V via USB	Supplies ESP32, sensor, and LEDs
Power — Motor	12V DC adapter	Supplies L298N and motor; isolated from logic rail

Firmware State Machine

The firmware operates as a two-state finite state machine (FSM) with a hysteresis buffer. The boolean variable windowOpen tracks the current mechanical position. State transitions occur only when the sensor reading crosses the threshold and the stability delay elapses. A second ADC read at the end of the delay confirms the condition before actuation.

State Transition	Condition
CLOSED → OPENING	gasValue < 1900 AND windowOpen == false, confirmed after 10 s
OPEN → CLOSING	gasValue >= 1900 AND windowOpen == true, confirmed after 10 s
Remain in current state	Condition does not persist through the stability gate
Motor stops	6500 ms of PWM drive elapsed; IN1 and IN2 both set LOW

06 Hardware Design

Bill of Materials

Component	Specification	Qty
ESP32 Development Board	240 MHz dual-core, 12-bit ADC, Wi-Fi/BT capable	1
MQ Gas Sensor Module	Analog output; 0–5 V; heater element, DO/AO pins	1
L298N Motor Driver Module	Dual H-bridge; 2 A/ch; 12 V motor, 5 V logic	1
12V DC Geared Motor	30 RPM, adequate torque for rack drive	1
Blue LED, 5 mm	Window-open indicator	1
Red LED, 5 mm	Window-closed indicator	1
220 Ω Resistors, 0.25 W	LED current limiting	2
Breadboard, full-size	830-point tie-strip prototyping board	1
5V USB power supply	Logic rail for ESP32, sensor, and LEDs	1
12V DC adapter (≥1 A)	Motor rail, isolated from logic supply	1
Jumper wires (M-M, M-F)	Circuit interconnects	Set

GPIO Pin Assignment

ESP32 GPIO	Connected to	Signal Type
GPIO 34 (ADC1_CH6)	MQ Sensor analog output	Analog input (0–3.3 V)
GPIO 26	L298N IN1	Digital output (motor direction A)
GPIO 27	L298N IN2	Digital output (motor direction B)
GPIO 25	L298N ENA	PWM output (motor speed, 0–255)
GPIO 18	Blue LED (220 Ω series)	Digital output (window-open indicator)
GPIO 19	Red LED (220 Ω series)	Digital output (window-closed indicator)
5 V	MQ Sensor VCC	Power rail (sensor compatible)
GND	Common ground reference	Shared between 5 V and 12 V rails

Power Isolation:
The 12V motor supply and 5V logic supply share only the GND reference. Operating both from the same supply caused the ESP32 to reset during motor start due to inrush current voltage drop. Separate supplies eliminated this issue entirely.

07 Mechanical Design

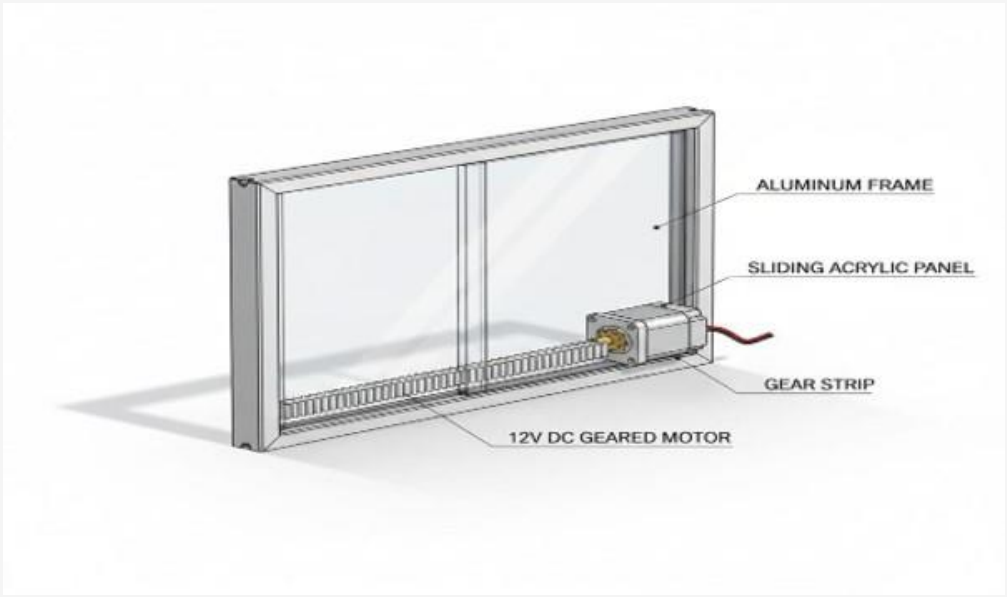


Figure 7.1: System Design Render — Window Assembly Exploded View

Technical Note: The motor body mounts directly to the bottom rail of the frame. Rigid side bracing was added during prototyping to prevent motor flex that caused pinion-to-rack disengagement under load.

Mechanical Specifications

Parameter	Value
Rack length	11 inches (279.4 mm)
Pinion diameter	3 cm (30 mm)
Motor speed	30 RPM
Motor travel time (firmware)	6500 ms
Window panel material	Acrylic (prototype)
Frame material	Aluminum extrusion
Drive type	Rack and pinion (linear)
Motor mounting	Bottom-rail fixed with rigid side bracing

Transmission Calculation

```
Pinion circumference  =  π × d  =  π × 30 mm  ≈  94.25 mm/revolution
Linear speed @ 30 RPM  =  (94.25 mm × 30) / 60  =  47.12 mm/s
Full stroke (11 inches)  =  279.4 mm
```

```
Theoretical travel time = 279.4 ÷ 47.12 ≈ 5.93 s → Firmware set
to 6500 ms (+10% margin)
```

Design Rationale: Travel Time Buffer

A 6500 ms drive time was selected rather than the theoretical 5930 ms to account for voltage sag under motor load and mechanical resistance variation. Stopping slightly past the full stroke is preferable to stopping short, which would leave the window partially open.

08 Electronics Design

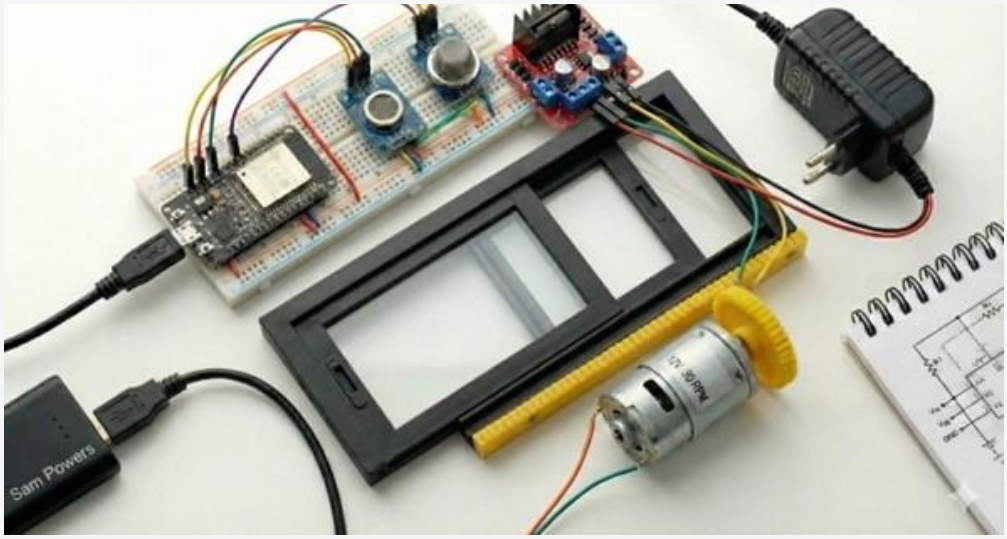


Figure 8.1: Hardware Prototype Setup — Complete Breadboard Assembly

Technical Note: The +/– power bus strips along the board edges serve as dedicated 5V and 12V rails. Motor-side wiring is kept physically separated from sensor and MCU wiring to reduce EMI coupling.

Motor Driver Logic Table

IN1	IN2	ENA (PWM)	Motor Action
HIGH	LOW	200 / 255	Forward rotation — window opens
LOW	HIGH	200 / 255	Reverse rotation — window closes
LOW	LOW	0	Motor stopped — coasting

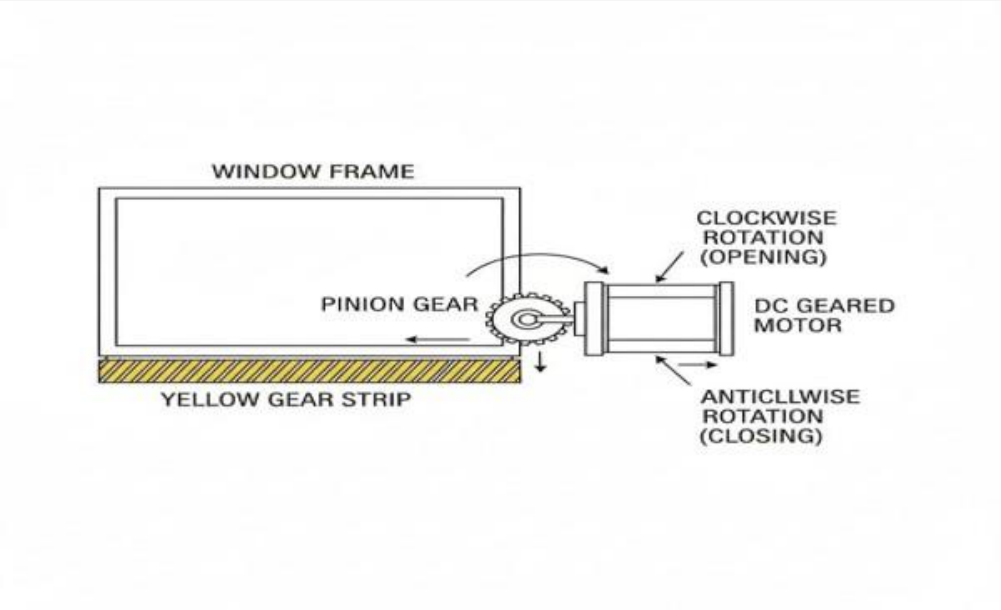


Figure 8.2: Rack and Pinion Mechanism — Drive Assembly Schematic

Technical Note: Clockwise rotation opens the window; anticlockwise closes it. Direction is controlled entirely in firmware via the L298N IN1/IN2 logic levels. The yellow gear strip engages the pinion directly on the bottom rail.

LED Indicator States

LED	State Meaning
Blue LED (GPIO 18) ON	Window is open — clean air condition confirmed
Red LED (GPIO 19) ON	Window is closed — polluted air condition confirmed
Both LEDs OFF	System initializing or motor actively driving

09 Software Architecture

Firmware Design

The firmware is written in Embedded C using the Arduino framework for ESP32. The main loop polls the gas sensor at 1-second intervals. A boolean variable (windowOpen) tracks the current window state. State transitions are gated by a 10-second stability delay, which requires a second ADC read at the end of the delay to confirm the condition before actuating the motor.

Final Firmware — Production Version

```
// ============================================================
//  Automated Sliding Window for Ambient Air Quality
//  Platform  : ESP32
//  Sensor    : MQ Gas Sensor
//  Driver    : L298N Motor Driver
//  Version   : v1.0 (Final)
// ============================================================

// --- Pin Definitions ---
const int gasSensorPin = 34;   // ADC1_CH6
const int IN1          = 26;   // Motor direction A
const int IN2          = 27;   // Motor direction B
const int ENA          = 25;   // PWM speed control
const int LED_BLUE     = 18;   // Window-open indicator
const int LED_RED      = 19;   // Window-closed indicator

// --- System Parameters ---
const int   GAS_THRESHOLD   = 1900;    // ADC units (12-bit, 0-4095)
const int   MOTOR_TRAVEL_MS = 6500;    // Full rack stroke time (ms)
const int   STABILITY_DELAY = 10000;   // Anti-oscillation gate (ms)
const int   SENSOR_INTERVAL = 1000;    // Polling interval (ms)
const int   PWM_SPEED       = 200;     // Motor speed (0-255)

bool windowOpen = false;

// --- Setup ---
void setup() {
  Serial.begin(115200);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(LED_BLUE, OUTPUT); pinMode(LED_RED, OUTPUT);
  stopMotor();
  digitalWrite(LED_RED, HIGH);  // Default: window closed
  Serial.println("System initialized. Sensor warming up...");
  delay(60000);  // Sensor warm-up period (60 s)
}

// --- Main Loop ---
void loop() {
```

```
  int gasValue = analogRead(gasSensorPin);
  Serial.printf("ADC: %d  Threshold: %d  Window: %s\n",
    gasValue, GAS_THRESHOLD, windowOpen ? "OPEN" : "CLOSED");

  if (gasValue < GAS_THRESHOLD && !windowOpen) {
    delay(STABILITY_DELAY);
    if (analogRead(gasSensorPin) < GAS_THRESHOLD) {
      openWindow();
      windowOpen = true;
    }
  }
  else if (gasValue >= GAS_THRESHOLD && windowOpen) {
    delay(STABILITY_DELAY);
    if (analogRead(gasSensorPin) >= GAS_THRESHOLD) {
      closeWindow();
      windowOpen = false;
    }
  }
  delay(SENSOR_INTERVAL);
}

// --- Motor Control Functions ---
void openWindow() {
  Serial.println("Opening window...");
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  analogWrite(ENA, PWM_SPEED);
  delay(MOTOR_TRAVEL_MS);
  stopMotor();
  digitalWrite(LED_BLUE, HIGH); digitalWrite(LED_RED, LOW);
}

void closeWindow() {
  Serial.println("Closing window...");
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  analogWrite(ENA, PWM_SPEED);
  delay(MOTOR_TRAVEL_MS);
  stopMotor();
  digitalWrite(LED_BLUE, LOW); digitalWrite(LED_RED, HIGH);
}

void stopMotor() {
```

```
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  analogWrite(ENA, 0);
}
```

Key Design Decisions

Double-Read Confirmation

The firmware re-reads the ADC after the stability delay before committing to actuation. This eliminates false positives from gradual sensor drift that may have slowly crossed the threshold during the 10-second window.

Sensor Warm-Up Delay

The MQ sensor heater requires approximately 60 seconds to reach thermal equilibrium. During this period, ADC readings are elevated and unreliable. The firmware holds in setup() for 60 seconds before enabling the control loop.

Serial Telemetry

Every loop iteration logs the ADC value, threshold, and window state at 115200 baud. This enabled real-time monitoring during calibration without additional hardware and was essential for determining the correct threshold value of 1900 ADC during field testing.

10 Engineering Calculations

Mechanical Transmission

Parameter	Value
Pinion diameter	30 mm
Pinion circumference (πd)	94.25 mm per revolution
Motor speed	30 RPM = 0.5 rev/s
Linear actuator speed	94.25 × 0.5 = 47.12 mm/s
Rack length (full stroke)	279.4 mm (11 inches)
Theoretical travel time	279.4 ÷ 47.12 = 5.93 seconds
Firmware travel time	6500 ms (10% safety margin applied)

Sensor Threshold Calibration

The threshold value was determined experimentally. The MQ sensor was monitored via Serial output under clean ambient air conditions for five minutes to establish the baseline ADC range. Incense smoke was then introduced at measured distances to observe the sensor response. The threshold of 1900 ADC units was selected to provide clear discrimination between ambient variation and a genuine pollution event.

Condition	Observed ADC Range
Clean ambient air (baseline)	1100 – 1400 ADC
Mild pollution near threshold	1700 – 1900 ADC
Dense incense smoke (10 cm from sensor)	2100 – 2800 ADC
Threshold set point	1900 ADC

Power Consumption Estimate

Component	Estimated Current Draw
ESP32 (active polling)	~80 mA @ 3.3 V
MQ Gas Sensor (heater active)	~150 mA @ 5 V
L298N (quiescent)	~36 mA @ 12 V
12V DC Motor (running)	~350–500 mA @ 12 V
LED indicators (each)	~10 mA @ 3.3 V
Total (motor active)	~650 mA @ 12 V + ~250 mA @ 5 V (approximate)

11 Prototype Development

Phase 1 — Problem Analysis and Component Selection

We surveyed available low-cost air quality sensing and ventilation control solutions. The ESP32 was selected over Arduino Uno for its 12-bit ADC resolution (4096 levels vs. 1024), which provides finer granularity in the threshold region. The L298N was selected for its integrated flyback diodes and ability to handle motor back-EMF without external components.

Phase 2 — Circuit Assembly and Initial Testing

We assembled the circuit on a full-size breadboard with dedicated power bus strips for the 5V logic rail and 12V motor rail. During initial single-supply testing, the ESP32 reset each time the motor started due to inrush current collapsing the supply voltage. Separating the supplies with a shared GND reference resolved this issue.

Phase 3 — Firmware Development and Threshold Calibration

We developed the initial firmware with a threshold of 1800 ADC, derived from datasheet estimates. Field testing with incense smoke revealed that 1800 ADC was too close to the upper end of the clean-air

baseline range, causing occasional false actuations. The threshold was raised to 1900 ADC. The stability delay was extended from an initial 3 seconds to 10 seconds after observing motor chatter during brief outdoor pollution events.

Phase 4 — Mechanical Assembly and Alignment

The rack and pinion assembly required precise motor mounting. An initial flexible mounting bracket allowed motor body flex under load, causing the pinion gear to partially disengage from the rack at stroke endpoints. Rigid aluminum side bracing was added to the motor bracket, and no rack skip events occurred after this modification.

Phase 5 — System Integration and Validation Testing

We tested the complete integrated system under controlled laboratory conditions using incense smoke introduced at measured distances. Testing covered clean-air conditions, polluted-air conditions, and boundary conditions near the threshold. The system demonstrated reliable and stable operation across all test scenarios.

12 Results and Validation

Test Conditions

Testing was conducted in a controlled indoor laboratory environment. Incense smoke was used as the simulated pollutant source. The sensor was mounted at window-sill height. Each test cycle involved introducing or clearing smoke and observing system response. The stability delay was allowed to complete fully before each actuation was confirmed.

Functional Test Results

Test Scenario	Sensor Reading	Expected	Observed
Normal ambient air	Below 1900 ADC	Window opens	Successful
Incense smoke introduced	Above 1900 ADC	Window closes	Successful
Brief spike (<10 s)	Above 1900, then drops	No actuation	Stable — held
Sustained pollution (>10 s)	Above 1900 sustained	Window closes	Successful
Fluctuation near threshold	~1850–1950 ADC	No actuation	Stable — held
Extended ambient monitoring	Below 1900 throughout	No change	No false actuations

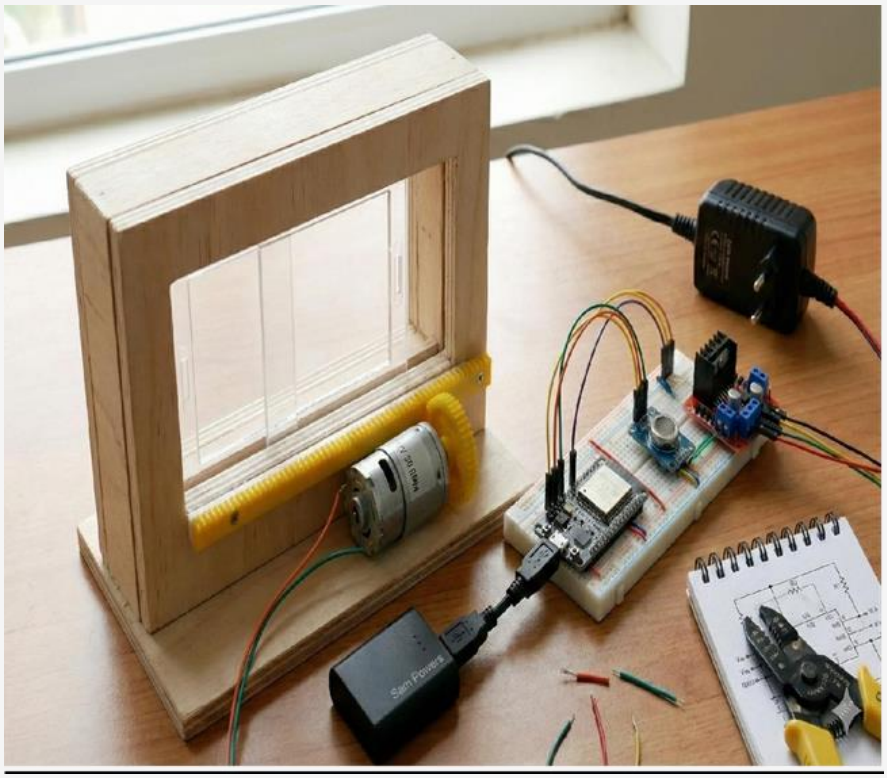


Figure 12.1: Final Prototype Model — Complete Integrated Assembly

Technical Note: The wooden frame prototype demonstrates the full mechanical and electronic integration: sliding acrylic panel, rack-and-pinion drive, DC motor, ESP32 control board, and sensor module.

Mechanical Performance

Metric	Observation
Full stroke time (measured)	6.2 – 6.4 seconds across multiple cycles
Rack engagement	No skip events observed after bracket bracing was applied
Motor temperature	Warm to touch after sustained operation; no thermal cutout triggered
Mechanical alignment	No drift or misalignment observed across test cycles

Sensor Stability

Under clean ambient conditions, the MQ sensor produced stable readings in the range of 1100–1400 ADC after the 60-second warm-up period. Readings were consistent over an extended monitoring period, with no spurious threshold crossings observed. The sensor responded promptly when smoke was introduced and returned to baseline after the smoke was cleared.

Note on Test Scope:
Testing was conducted at prototype scale under laboratory conditions. Results demonstrate functional correctness of the design. Field deployment in varied environmental conditions would require additional calibration of the threshold value based on site-specific baseline sensor readings.

13 Challenges and Lessons Learned

Challenge 1 — Single-Supply Voltage Collapse

Operating the ESP32 and motor from a single 5V supply caused the ESP32 to reset every time the motor started. The DC geared motor draws significantly higher current during startup than during steady-state operation, causing the supply voltage to momentarily drop below the ESP32 operating threshold.

Resolution:
The motor supply was separated to a dedicated 12V DC adapter. The two supplies share only the GND reference. This eliminated all reset events during motor startup.

Challenge 2 — MQ Sensor Warm-Up Drift

The MQ gas sensor produces unstable and elevated ADC readings during the first 60 seconds after power-on while the internal heater element reaches operating temperature. In early firmware builds without a warm-up delay, the system immediately closed the window on boot due to these elevated startup readings.

Resolution:
A 60-second blocking delay was added at the end of setup() before the main control loop begins. Serial output during this period indicates the warm-up state to the user.

Challenge 3 — Rack Skip Under Motor Load

The initial motor mounting used a single-point bracket that allowed the motor body to flex slightly when the sliding window panel encountered mechanical resistance. This flex caused the pinion gear to partially disengage from the rack teeth, producing an audible skip and incomplete window displacement.

Resolution:
Rigid aluminum side bracing was added to both sides of the motor body, constraining all axes of flex. No rack skip events occurred after this modification.

Challenge 4 — Threshold Selection Under Unit Variation

MQ-series sensors exhibit significant unit-to-unit variation in absolute output level due to manufacturing tolerances. The datasheet threshold estimate of 1800 ADC was found to be unreliable for the specific sensor unit used, placing the estimate uncomfortably close to the observed clean-air noise floor.

Resolution:
A calibration protocol was established: read the sensor baseline in clean air for five minutes, record the mean ADC value, and set the threshold at mean + 500 ADC units. For the specific sensor used, this yielded a threshold of 1900 ADC.

14 Applications

Potential Deployment Environments

Environment	Application	Benefit
Residential homes	Bedroom and kitchen windows	Passive overnight air quality management
Hospitals and clinics	Patient ward ventilation	Reduced pollutant exposure for vulnerable occupants
School classrooms	Ventilation during outdoor events	Maintains acceptable CO_2 and AQI for learning environments
Research laboratories	Fume-aware ventilation control	Automatic closure during chemical or fume events
Office buildings	Multi-zone ventilation layer	Integration with building management systems
Industrial facilities	Worker safety — variable-pollution zones	Supplemental actuation during process-generated events

Scalability:
The core design requires one ESP32 per controlled window. In multi-window configurations, a central sensor can distribute readings to multiple ESP32 units via I2C or Wi-Fi, reducing per-window sensor cost.

15 Future Scope

Enhancement	Technical Approach	Priority
IoT monitoring dashboard	ESP32 Wi-Fi + MQTT to Node-RED or Home Assistant	High
Remote manual override	Mobile app control via Blynk or custom interface	High
Multi-gas sensing	Replace MQ with BME688 (PM2.5, CO_2, VOC, temperature, humidity)	High
Area AQI integration	Supplement local sensor with AQICN / OpenAQ API data	Medium
Solar power supply	Photovoltaic panel with Li-ion buffer for off-grid use	Medium
OTA firmware updates	ESP32 ArduinoOTA for threshold adjustment without reflashing	Medium
Adaptive threshold	Rolling baseline model to auto-adjust threshold to site conditions	Future

Architecture Principle for v2

The actuation loop should remain local and deterministic regardless of cloud connectivity. Cloud features should be additive for logging, monitoring, and configuration — the window should continue to operate correctly even when the internet connection is unavailable.

16 Team Contributions

Member	Role	Contributions
Samarth G. Ichageri 2JH25EC033	Electronics & Mechanical Systems Lead	Circuit schematic design, ESP32 firmware development, threshold logic, motor control, full system integration, Sliding window frame fabrication, rack-and-pinion assembly, motor mounting bracket design and bracing
Satvik Pandurangi 2JH25EC035	Hardware & Software Validation Engineer	Breadboard assembly and wiring, power supply configuration, gas sensor calibration, component testing, Algorithm review, code debugging, test execution, validation documentation

17 References

[1] Espressif Systems. ESP32 Technical Reference Manual. Espressif Systems, 2023.

[2] Hanwei Electronics. MQ Gas Sensor Series Datasheet. Zhengzhou Winsen Electronics Technology Co., Ltd., 2022.

[3] STMicroelectronics. L298 Dual Full-Bridge Driver Datasheet. STMicroelectronics, 2021.

[4] Arduino. Arduino IDE Documentation. Available: https://www.arduino.cc/en/software

[5] World Health Organization (WHO). Air Quality Guidelines: Global Update 2021. WHO Press, Geneva, 2021.

[6] J. Fraden. Handbook of Modern Sensors: Physics, Designs, and Applications. 5th ed. Springer, 2016.

[7] K. Ogata. Modern Control Engineering. 5th ed. Pearson Education, 2010.

[8] IEEE Xplore Digital Library. Selected research articles on IoT-based Air Quality Monitoring Systems.

18 Repository Structure

The project repository is organized to allow engineers and contributors to locate firmware, hardware documentation, and mechanical reference material independently.

```
automated-sliding-window/
│
├── firmware/
│   └── main.ino                                  # Final ESP32 firmware implementing air-quality
│                                                   based automatic window opening and closing logic
│
│
├── docs/
│   ├── Project_Report.docx                       # Complete project report
│   │
│   └── Project_Presentation.pptx                 # Project presentation slides
│
├── images/
│   ├── System_Design_Render.jp                   # 3D system concept rendering
│   │
│   ├── Hardware_Prototype_Setup.jpg              # Hardware implementation and electronics setup
│   │
│   ├── Rack_and_Pinion_Mechanism.jpg             # Mechanical rack-and-pinion actuation mechanism
│   │
│   ├── Circuit_Diagram.jpg                       # Circuit wiring and pin connections
│   │
│   └── Final_Prototype_Model.png                 # Final assembled prototype
│
└── README.md                                     # Project overview, hardware requirements,
                                                    wiring guide, setup instructions and usage
```

Getting Started

1. Clone the repository. Open `main.ino` in Arduino IDE with the Espressif ESP32 board package installed.
2. Wire the hardware per `Circuit_Diagram.jpg` Ensure the 5V and 12V supplies are connected to separate rails.
3. Run `main.ino` for five minutes in clean air. Note the mean ADC value and set GAS_THRESHOLD = mean + 500 in `main.ino`
   Flash `main.ino` to the ESP32. Open Serial Monitor at 115200 baud to observe ADC readings, threshold comparisons, and state changes during the 60-second sensor warm-up and subsequent operation.